

Finding 1 — most "outliers" carry real signal

I compared the default rate above the 99th percentile against everyone else:

feature	n	default rate	vs rest
dti > 39	5,103	6.41%	+2.74 pp ← riskier
delinq_amnt > 0	2,510	5.06%	+1.37 pp ← riskier
revol_util > 99	5,059	4.80%	+1.12 pp ← riskier
annual_inc > 270k	5,041	2.46%	-1.25 pp ← safer
total_rev_hi_lim > 165k	5,127	2.28%	-1.43 pp ← safer
revol_bal > 105k	5,133	2.44%	-1.27 pp ← safer

Every one of those points somewhere. Capping at the 99th percentile would flatten all six. **Your source's point 2 beats its point 3, and your own data is the evidence.**

Finding 2 — the signal reverses in the impossible tail

dti > 39	(5,103 rows)	6.41% default	← well above base rate
dti ≥ 100	(81 rows)	1.23% default	← well BELOW base rate
dti = 999	(5 rows)	0.00% default	

Risk climbs with `dti` — then collapses. Real financial stress doesn't behave like that. That collapse is the fingerprint of junk data, and it draws the line for you: somewhere above 39 the column stops describing borrowers.

Also worth knowing: the IQR method is useless on this data. It flags 94,264 rows of `pub_rec` (18% of your data) purely because the median is zero. Zero-inflated count columns break it.

The table

Feature	Flagged range	Possible explanation	Error or signal?	Investigation needed	Possible treatment	Risk of capping
<code>dti</code>	<code>= 999</code> (5), <code>< 0</code> (1)	Placeholder / impossible	Error	None — 999 is not a ratio	Correct or remove	None
<code>dti</code>	<code>100-998</code> (76)	Junk; signal reverses here	Probably error	Check their <code>annual_inc</code> — one had \$1,000	Correct or remove	Low
<code>dti</code>	<code>39-100</code> (5,022)	Genuine over-indebtedness	Signal — 6.41% default	None	Keep	High — destroys your strongest risk marker
<code>annual_inc</code>	<code>= 0</code> (36)	LC requires stated income	Error	Why did 36 pass underwriting?	Correct or remove	None
<code>annual_inc</code>	<code>< 5,000</code> (60)	Student? Retiree? Typo?	Unclassified	Compare to <code>loan_amnt</code> — could they service it?	Decide after checking	Low
<code>annual_inc</code>	<code>> 1m</code> (119)	Genuinely wealthy	Signal — 3.36%, near base	None	Keep	Moderate — loses real high earners
<code>total_rev_hi_lim</code>	<code>= 9,999,999</code> (1)	All-9s placeholder	Error	None	Correct or remove	None
<code>total_rev_hi_lim</code>	<code>> 165k</code> (5,127)	High credit limits	Signal — 2.28%, protective	None	Keep	High
<code>revol_util</code>	<code>> 150%</code> (9)	Beyond plausible over-limit	Probably error	Check against <code>revol_bal</code> ÷ limit	Correct or remove	None
<code>revol_util</code>	<code>100-150%</code> (2,331)	Over their limit — real	Signal — 4.66% default	None	Keep	High

Feature	Flagged range	Possible explanation	Error or signal?	Investigation needed	Possible treatment	Risk of capping
<code>credit_history_months</code>	<code>> 840</code> (3)	Implies a 100-year-old	Error	Already checked: <code>earliest_cr_line</code> = 1933	Correct or remove	None
<code>credit_history_months</code>	<code>600-840</code> (552)	50-70 year histories	Unclassified — 7.03% default, <i>double</i> base	Are these elderly borrowers, or bad dates?	Investigate first	Moderate
<code>pub_rec</code>	<code>> 20</code> (15)	61 bankruptcies for one person	Unclassified	Bureau reporting quirk? All 15 defaulted 0%	Investigate	Low
<code>tax_liens</code>	<code>> 20</code> (14)	Same pattern	Unclassified	Same	Investigate	Low
<code>delinq_amnt</code>	<code>> 0</code> (2,510)	Currently overdue	Signal — 5.06%	None	Keep	High
<code>tot_coll_amt</code>	<code>> 5,672</code> (5,132)	Debts sent to collectors	Signal — 3.02%	None	Keep	Moderate

Total flagged as probable error: about 260 rows out of 513,300 — 0.05%. That's two orders of magnitude smaller than the ~10,000 rows per column that 1st/99th capping would have touched.

I'm not deciding any of these. Each one is your call.

Which single column would you like to investigate first — and what's your own default assumption about its extreme values?

My suggestion for where to start, though it's yours to overrule: `credit_history_months` in the 600-840 range. It's the only group in the table defaulting at nearly twice the base rate while sitting in the "can't classify" bucket, so it's where investigation would actually change something.